CyPhyHouse: A programming framework for rapid development of distributed mobile robotic applications

Ritwika Ghosh, Joao Paulo Jansch-Porto, Peter Du, Daniel Hernandez, Sayan Mitra, and Geir Dullerud
{rghosh9,janschp2,peterdu2,olivash2,mitras,dullerud}@illinois.edu
Coordinated Science Laboratory
University of Illinois at Urbana-Champaign
Urbana, IL 61801

***Abstract*— This paper introduces a new programming and software development framework—CyPhyHouse—for distributed robotic applications. The framework is motivated by the need for programming languages, compilers, simulators, debuggers and verification tools that are on par in their efficiency and effectiveness with those available for purely cyber software development. We introduce our development framework and the programming language. Through example applications, we overview the key development tools, the overall CyPhyHouse workflow for deploying code on heterogeneous mobile platforms, and we discuss the technical challenges and design decisions.**

I. Introduction

Consider the development process for a location-sharing application for mobile phones. The developer codes in a high-level language like Objective C or Swift, focusing her energies on getting the application logic right. A compiler generates the executable code automatically. Even executables for different hardware platforms can be generated automatically, essentially by linking the correct platform-specific libraries. The programming and compiler infrastructure comes with tools for testing, debugging, and validation. Because of this software framework, the simple app can be written, deployed, and tested in an afternoon.

Contrast that with the development of a similar location-based application for a heterogeneous collection of mobile robots. There are no languages for programming robots that give the programmer the high-level of abstractions to say, for example, "move from A to B avoiding U". The popular software ROS [1] provides many useful packages that work across different platforms, but it does not provide needed programming abstractions nor the compiler support. There is also a paucity of tools for deploying the generated code on the actual robots. Integrated, testing and validation tools for distributed robotic systems are also currently not available. Consequently, developing and debugging the application may take weeks. Since heterogeneous robotic platforms can be used for many tasks like mapping, surveillance, delivery, coordination, the absence of the development ecosystem means that each application has to be developed from scratch with little reuse, modularity, or portability. In this paper, we address these inefficiencies by introducing *CyPhyHouse*[1]: an open source software framework for programming, rapid deployment, and testing of distributed robotics applications. Using CyPhyHouse, a user can code, compile, launch, and run applications in a highly-automated fashion (Figure 1). The technical contributions of this paper are:

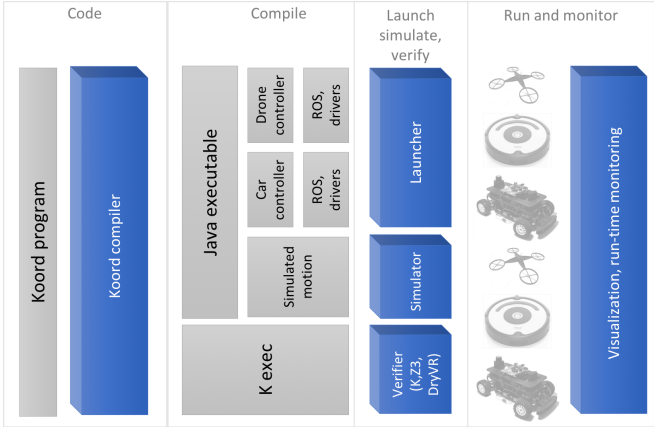

Fig. 1: CyPhyHouse framework. Major tools shown in blue.

- Design and development of the CyPhyHouse open source software system. This includes the simulator, the application launcher, the run-time logging and monitoring system, and an integrated indoor positioning system. All of these software tools are integrated with our new robot programming language called Koord and its compiler.
- Demonstration of the feasibility of end-to-end high-level application development and deployment on a distributed platform involving multiple quadcopters.
- Demonstration of the feasibility of system-level verification of applications written in Koord.

Design and development of the Koord programming language and the supporting verification tool KoordBMC, are discussed here for the sake of completeness; however, their development is a significant independent effort which is reported elsewhere [2].

Related Work: Many Domain Specific Programming Languages (DSLs) aim to automate development of robotic systems [3]. Barring a few exceptions, most of the older languages are proprietary or generate executables that are

[1] https://cyphyhouse.github.io

tied to specific platforms. There is a renewed interest in open and portable languages that raise the level of abstraction. The Buzz [4] and React [5] fall in this category as does our language *Koord*. The Live Robot Programming language [6] not only provides a higher-level programming abstraction in terms of nested state machines, it also allows the program to be changed while it is running, and hence, reduce the feedback loop across writing, compiling, and testing of robot programs. Buzz currently does not connect with verification tools. The verification approach implemented with React uses precise models of the environment and performs model checking using dReal [7]. In contrast, our environment implementations may not have precise models and our verification approach relies on sensitivity analysis [8]. Our approach is also similar in spirit to the Reactive Model-based Programming Language (RMPL) [9]. In contrast to providing users with languages and development tools, the Robotarim project provides remote access to a full-fledged multi-robot research facility [10]. The VeriPhy project [11] shares a similar goal to CyPhyHouse; but, instead of a programming language the starting point is differential dynamic logic [12] and there are significant differences in techniques used (KeYmaera X, HOL instead of K, Z3, DryVR).

There is a huge body of research on "correct-by-construction" synthesis of controllers from temporal logic specifications (see, for example [13], [14], [15], [16], [17]). Many of these approaches have been applied to mobile robotic systems. Our point of view for programming robots is fundamentally different in what we see as the role of the user. A correct-by-construction synthesis algorithm takes as input a high-level requirement (for example, "from room A to B and see if you find a chair") to generate robot programs for accomplishing this task. In our approach, programmer's creativity is still required in writing the Koord program that solves the problem, but the tedious and error-prone steps in low-level coordination and control are automated.

In a different cluster, are the programming systems using the shared memory paradigm [18], [19], [20], [21], [22]. Our *Koord* language provides distributed shared variable abstraction for coordinating multiple agents. The consistency semantics implemented here is that the writes to shared variables are propagated to all the agents, and become visible to other agents reading the variable after one round (δ time units). In the fault-free synchronous model considered here, any number of gossip-based algorithms can be used to implement this semantics in the *Koord* runtime system. P [23] and PSync [24] are DSLs for asynchronous partially distributed systems, but cyber-physical interactions are not supported in these languages.

II. Overview of Building Apps using CyPhyHouse

Building a new CyPhyHouse application involves four easy steps: code, compile, launch, and run. In what follows we describe the software suite we have developed for supporting each step.

A. High-level coding for robotics with Koord

The user writes the application code using our new high-level programming language: *Koord*. We have developed Koord as an event-triggered, distributed shared memory-based programming language that provides abstract constructs for robot motion control. The Koord program for a set of 10 robots to form a equi-spaced line is shown in Figure 2. Notice how close this program is to a textbook description of a consensus protocol: Robot pid periodically sets its destination to be the average of the positions of $pid - 1$ and $pid + 1$. If all robots (except the extremal ones) repeat this operation periodically, then it is well-known that they would eventually form an equi-spaced line. Thanks to the shared variable x that Koord allows to declare in our *Lineform* program, the positions of the neighboring robots are available to read at robot pid, which in turn makes this program succinct yet understandable.

```
Lineform
  using Motion actuator target sensor pos, status
  allread: Position ⟨∗⟩ x

  Update:
    eff x[pid] = Motion.pos
      if (pid ≠ 1 or pid ≠ 10)
        Motion.target = (x[pid + 1] + x[pid − 1])/2
```

Fig. 2: Line formation program written in *Koord* (*Left*) and its mathematical counterpart in robotics and control textbooks (*Right*).

Lineform is a simple application. In Section III we discuss a more interesting application (*Task*) in which a collection of robots have to coordinate and visit a sequence of points in a mutually exclusive and exhaustive fashion. We imagine this application to be a building-block for, for example, collaborative mapping, delivery, inspection, fire-fighting. Other applications we have written using Koord include a application for a smart intersection, a distributed landing protocol inspired by NASA's SATS [25], and a platooning application for automated vehicles. For a deep dive into syntax, semantics, and pragmatics of Koord, we have the language design document [2].

B. Compiling to run, simulate, or verify applications

We have developed a Koord compiler that automatically generates executables from the high-level application code (Figure 2), which can be deployed on different platforms. Currently implemented support includes quadcopters, Roombas, and RC cars. The compilation process has a few stages. First, a (platform independent) Java executable is generated. This executable is then linked with platform dependent libraries such as ROS drivers for messaging and low-level controllers. For example, different path-planners and low-level controllers are linked for quadrotors, than those that are used for RC cars. The resulting linked executables are then ready to be uploaded on the individual robots.

The platform independent executable can also be simulated using the *CyPhyHouse simulator* (see, Figure 3). The

simulator runs the actual executable but simulates the motion of the robots according to specified physics models. It is highly customizable, provides 2D-visualization, and provides a clear and quick way of testing the application on a large number of heterogeneous robots.

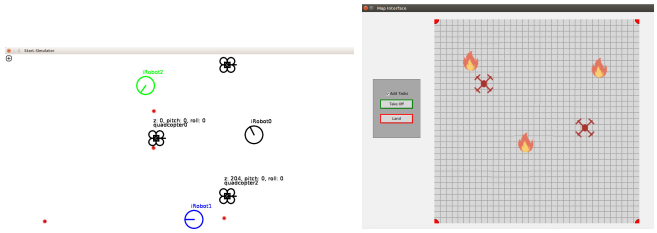

Fig. 3: *Left:* CyPhyHouse simulator screenshot. *Right:* User interface for run-time tracking of CyPhyHouse applications deployed on robot platforms.

Finally, the we have developed the complete formal semantics of Koord in the $\mathbb{K}$ framework [26]. This means that we can formally verify the high-level application code against required invariant properties. For example, the *KoordBMC* tool we have developed can automatically check that the system of robots executing the $Lineform$ application satisfies the invariant "$\forall i \in ID, x[0] \leq x[i] \leq x[10]$" which asserts that all the robots stay within the segment defined by robots 0 and 10. Verification of distributed systems is a fundamentally challenging problem with very few tools and algorithms that scale [27]. Verification of applications controlling continuous motion makes this into a hybrid verification problem, which is even more challenging. Our verification approach combines the formal semantics of $\mathbb{K}$ with state-of-the art hybrid verification tool DryVR [8] to show, for the first time, the feasibility of code-level verification of distributed hybrid systems. In Section III we present a summary of our verification techniques. Once again, we refer the reader to [2] for technical details.

C. Automatically launching on heterogeneous platforms

Without automation, the mundane task of compiling and running programs can become arduous when many devices are involved. It does become absolutely unmanageable and error prone when different types of robot platforms have to get different versions of the executable, linked with correct libraries and IP addresses. Our *Launcher* program makes this a one step automatic process. It registers all the participating robots, compiles their application codes (using the above compiler and the correct platform dependent libraries), loads the executables, sets up the indoor positioning system, and starts the application execution. In general, computing the membership of agents in a group of asynchronous processes is a well-studied hard problem [28]. Launching also involves the practical issue of discovering the IP addresses of the agents in the group. The implementation of our Launcher protocol is discussed in Section III.

D. Real-time monitoring and running applications

When the application is executing on the platforms, the CyPhyHouse runtime framework records detailed timed logs including program states, events, positions, sensor data, message queues, etc. These logs can be visualized and annotated for the purpose of debugging. We have also developed a simple 2D, visualization for real-time monitoring and for users to provide inputs to the applications (Figure 3).

III. Key Software Modules in CyPhyHouse

In this section, we discuss the design of key software components in Figure 1.

A. Koord language and compiler

Koord is an event-driven programming language. It supports coordination across robots through distributed shared memory (DSM) and special motion control primitives. A $Koord$ program uses *sensor* and *actuator* ports to communicate with the robot's hardware platform (see Figure 1). For example, the $Lineform$ application uses the $Motion.target$ to set a new destination (according to the configured coordinate system). Then, the platform-specific path planning and low-level controllers attempt to reach this target[2]. Sensor ports are used by the application program to read data from the platform. For example, (i) the $Motion.pos$ port gives the agent's position (for example, from our UWB indoor positioning system), and (ii) $Motion.status$ indicates whether the $Motion$ controller is active, idle, or failed.

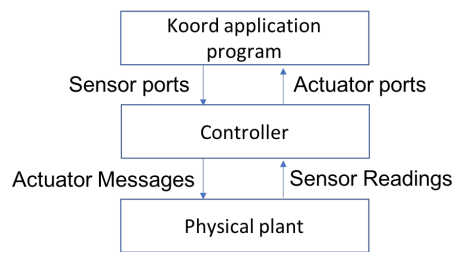

Fig. 4: A small $Koord$ application program and its interfaces to the execution platform.

A $Koord$ program consists of *variables declarations* and *events*. Variables can be local or shared. Distributed shared variables are implemented by $Koord$'s runtime system. As we saw earlier, $Lineform$ uses a shared array x of type $Position$. It is declared as $allread$, which means that the array $x[]$ has an entry for each robot id, all agents can read from $x[i]$, but only agent with $pid = i$ can write to $x[i]$. An $allwrite$ variable gives read/write access to all agents. Read and writes to shared variables are implemented using message passing by Koord's runtime system. The user writes the events in the precondition-effect style that is usual for concurrent programs (see Figure 5). Events define how program variables are updated. $Lineform$ uses a single $Update$ event, which sets the $target$ of each agent (except the extremal agents) to be the center of the position of its neighbors.

The *Task* application mentioned in Section II is shown in Figure 5. That the goal of this $Task$ application is to

[2]We have implemented RRT-based planners. Both the drones and the RC car use a simple PID controller.

use a set of robots to complete a shared list of spatially distributed tasks. Each task here is to visit a particular location in 3D space. In a more realistic version of this application, the task could involve visiting the point *and*, for example, making a sensor measurement, taking a picture, or picking-up an object. The robots have to cooperatively and mutually exclusively choose tasks, visit the location (without collisions), and then move on to the next task.

Task uses shared memory to store the sequence of points (in a list of positions called *dests*) for the robots to visit, and a shared index (*taskIndex*) to traverse the list in order. The **atomic** keyword is used to ensure no other robot has read or write access to the variables involved in the **atomic** block. The **atomic** keyword is then used to perform the safe addition of the tasks in the event Add, the mutually exclusive traversal of the list of destinations in Assign, and status update of a completed task (or a visited waypoint) in *Finish*. The Reached event queries whether a quadcopter has reached a task location that was assigned it.

Koord runtime system implements a synchronous round-by-round execution of events for all agents. That is, for a given execution parameter $\delta > 0$, one event per agent can occur every δ time. This type of synchronous update rule of Line 8 in *Lineform* is a typical example of a large family of textbook algorithms for distributed consensus, rendezvous, optimization, flocking, and pattern formation [29], [30], [31]. The user can specify a total running time (t), which would ensure that the runtime system repeatedly executes events with preconditions evaluating to true until t units of time have elapsed. Otherwise, the program could potentially execute indefinitely.

```
1  Task
2  using UI
3     sensors:
4        pos point
5        bool newTask
6     actuators
7        bool eraseTask
8
9  using Motion
10    sensors:
11       pos position,bool done
12    actuators:
13       pos target
14
15 allwrite:
16    list<pos> taskList
17    int taskIndex
18
19 local:
20    int status = 0
25 Add:
26    pre true
27    eff atomic add(UI.point,UI.newTask)
28
29 Assign:
30    pre status == 0 and ¬(Emty(taskList))
31    eff atomic
32       Motion.target = get(taskList,taskIndex)
33       taskIndex = taskIndex + 1
34       status = 1
35
36 Reached:
37    pre status == 1 and Motion.done
38    eff status = 2
39
40 Finish:
41    pre status == 2
42    eff atomic
43       status = 0
44       UI.eraseTask = true
```

Fig. 5: *Koord* program for coordinated *Task* app.

The *Koord* front-end contains a compiler that takes as input programs written in the *Koord* high-level language and generates Java application code that runs with the Java middleware for the Koord framework.

TABLE I: **Verification benchmarks summary**

Benchmark	Agents	δ (s)	Time Horizon (s)	Time to Verify (s)	Safe
Lineform	5	10	40	28.113	Yes
Fischer	2	1	10	479.931	Yes
SATS	3	50	200	38.382	Yes
Task	4	50	200	381.241	Yes
Smart-Intersection	3	3	15	131.423	Yes
Platooning	4	3	15	493.112	Yes

B. *KoordBMC for verification*

KoordBMC is a tool for checking bounded invariants (n-invariants) for *Koord* applications using explicit state reachability analysis. It works with both black-box and explicit environment models. The input to *KoordBMC* is (i) P: the program, (ii) inv: a candidate invariant function, (iii) ID: the set of agents, (iv) δ: time step size and (v) t_{max}: the time horizon for checking. *KoordBMC* uses the *Koord* executable semantics to produce all states that the system reaches during program transitions. It uses the DryVR [8] tool which over-approximates the reachable states of the sensor variables for a broad class of white-box and black-box models by learning the sensitivity of the trajectory of the sensor variables with respect to the set of states. DryVR gives a probabilistic guarantee about the correctness of the learned sensitivity function[3], and assuming that the sensitivity is correct it soundly over-approximates the set of reachable states from a given state. The tool outputs 'safe' if the inv is indeed an invariant (for the bounded execution), 'unsafe' if it finds a counter-example within the bounds of the execution. In some cases, it can return 'unknown' as the over-approximation of reachable states may not be refined enough to conclude whether the application is safe or unsafe.

A sampling of verification results are shown in Table[4]. The *Agents* column in the table refers to the number of agents we performed the experiments with, δ is the time increment during controller transitions, the *Time Horizon* column is the amount of time the application is run. The time taken for the verification takes into account the explicit state bounded model checking using $\mathbb{K}$, and the sensitivity analysis.

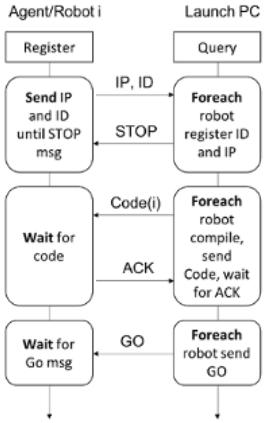

Fig. 6: Launch Protocol State Machine.

C. *Launch protocol*

One of the assumptions that the *Koord* application design currently makes is that each agent is aware of every agent in the system. To support this, we built upon directoryD [32], a software package that uses a decentralized architecture to automatically discover devices on the same WiFi network.

[3]The paper shows that 20-40 traces are enough to learn sensitivity of common nonlinear systems with nearly 100% accuracy.

[4]The computer used is a Intel Core i7-4960X CPU 3.60GHz and DDR3 RAM 64GB

Figure 6 represents the state machine corresponding to this process. The discovered IPs are then used by the code generation to automatically populate the required IPs to setup WiFi communication among the agents and the positioning system, and other (user interface) sensor nodes if required by an application.

D. Software-Hardware Bridge

The *Koord* runtime system has a three-plane architecture comprising of a program which interacts with the controller plane through sensor and actuator ports, which in turn communicates with the physical plant (in our case) through ROS [1] messages. The program plane itself consists of the high-level language interface with middleware that provides the language features. In a nutshell, the runtime system automatically generates code for the application using all the supported libraries, such as the implementation of the distributed shared variables using message passing over WiFi, motion automata of the robots, high-level collision and obstacle avoidance strategies, etc. We currently use a Java 1.8 implementation of the middleware, and we implemented the automatic code generation in Python 2.7+. Figure 1 includes the detailed structure of the software stack including the interfaces.

We use ROS to handle the low-level interfaces with hardware and as stated above, Java to implement the middleware. Therefore, to communicate between the high-level programs and low-level controllers, we need a means of communication between Java and ROS.

The Java middleware programs need to subscribe to existing ROS topics and execute appropriate callback functions upon receiving newly published messages. They should also be able to create ROS topics and publish messages to all existing subscribers. To this end, we use an object called the *JavaRosWrapper*, which is built upon the *Java Rosbridge* library [33] which connects Java code to a ROS Bridge server [34]. The *JavaRosWrapper* ensures that the Java program can access the message data from a Java object without having to worry about compatibility with ROS and vice versa.

E. Hardware Platform Positioning

So far, we discussed the hardware independent components of the *Koord* framework software stack; we now discuss the interaction of the quadcopter platform with the software stack and the positioning system used for localization.

The quadcopter stabilization and reference tracking are handled by Ardupilot [35]. Between the Java layer and Ardupilot we include a hardware abstraction layer (HAL) to convert setpoint messages from the high-level language into MAVLINK, using the mavROS library ([36]), so Ardupilot can parse them. Since the autopilot was originally meant to use a GPS module, we also convert the current quadcopter position (from either Vicon or UWB) into the Geographic Coordinate System, before sending it to the controller. We discuss the localization systems used by us in the next subsection. We also developed similar systems for two other platforms: RC cars and Roombas.

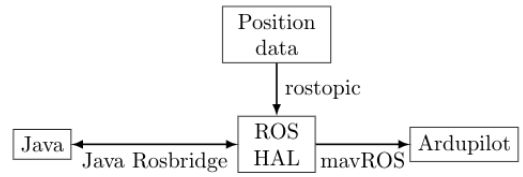

Fig. 7: Diagram of the interaction of the *Koord* software stack with the quadcopter. The ROS HAL node parses both position and setpoint messages into the message format that the Ardupilot uses.

F. Localization

In our testbed, aside from the existing Vicon setup, we are also implementing a localization system using ultra-wideband (UWB) technology. The UWB units are capable of measuring time-of-flight between other UWB units, enabling us to estimate the position of robots relative to fixed transmitters distributed inside the tested. We have 8 fixed UWB transmitters (also referred to as *Anchors*) transmit messages at a fixed time interval. A receiver in each robot (called *Tag*) calculates the time-difference of arrival (TDOA) between consecutive messages, and estimates its current position using an Extended Kalman Filter (EKF). Since each Tag is only receiving messages and estimating its position locally, we can track infinitely many robots at the same time.

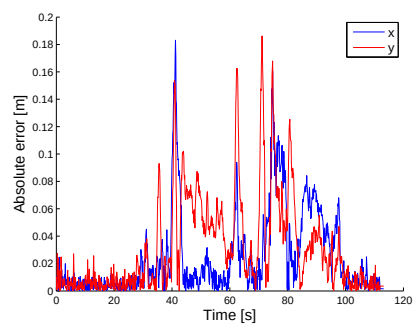

Fig. 8: UWB positioning system errors.

We can see a comparison of the performance of UWB as compared to the ground-truth Vicon data in Figure 8. Here we observe that the errors are, on average, below 10 cm with a few spikes below 20 cm. We note that the EKF filter implemented here only uses the TDOA messages and a simplified model of the quadcopter to estimate the position. If we added additional sensors to the filter, such as an accelerometer, and used a more accurate model of the dynamics, we would get even smaller errors.

IV. Experimental Validation

In this section we will discuss our experience in developing the Task app of Figure 5 using CyPhyHouse. Our main claim:*Koord language and the supporting CyPhyHouse*

tool suite enable distributed coordination applications to be developed, tested, and deployed effectively in a hardware independent fashion. We performed several experiments involving simulations and deployment of the $Task$ application on quadcopters and RC cars, and report our key findings.

High-level programming supported: The Koord supported programming the $Task$ app at a high-level of abstraction. The application programmer did not have to know about the low-level controller for the RC car or the quadcopters. Indeed, once the common software-hardware interface was designed, she developed the application program without interacting hardware platforms.

Compiler and Launcher work as expected: The Koord compiler generated the executables and the Launcher correctly loaded platform specific linked executables to each of the quadcopters. ries from one of the runs of the $Task$ application on two quadcopters is shown in Figure 9. *Left*. We also show the x coordinate vs $time$ plot annotated with the *assign* events for clarity in Figure 9: *Right*. The robots traversed the set of waypoints we sent to them mutually exclusively, and from the annotations on the x-trajectories of the drones, we can see that each quadcopter goes to the x-coordinate of the location of the task it was assigned to immediately after the *Assign* event is triggered. While the

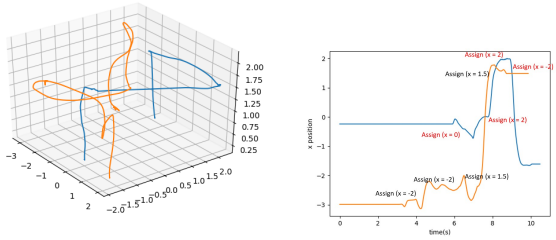

Fig. 9: *Left:* Simultaneous Trajectories of the quadcopters. *Right:* Variation of x coordinates of both quadcopters with time.

quadcopters executed $Task$, the UI tool (Figure 3) was useful for both real-time tracking of the drones and for providing the input task list (list of locations) to the $Task$ application.

Delay assumptions validated: For the shared memory abstraction to be faithfully implemented on the message-passing wireless network, the message delays between agents should be less than the the periodic round delay parameter δ (300ms). We observed from the experiments that the cumulative message delays for the quadcopter platform to be well below 100ms, and therefore, the executions were faithful to the semantics of the language.

Simulations help with large-scale performance evaluations and debugging: We were able to simulate the code for $Task$ using the Koord simulated in many different scenarios and with heterogenous participating robots. These simulations were useful for debugging and performance evaluation of the application with larger number of robots or tasks, and longer time-horizons.

Safety verification works for smaller instances: Verification of distributed cyber-physical systems is a challenging problems and most approaches do not scale well. Nevertheless, the *KoordBMC* tool could verify safety (collision freedom) of the $Task$ application for instances of the system with up to 5 agents and a short time horizon. We derived the quadcopter dynamics assuming a rigid body quadcopter using Newton-Euler equations. We implemented a simulator for the quadcopter flight data in Java, and added it to the executable language semantics as a blackbox.

Manual effort: While the CyPhyHouse tools automate many of the application development steps, certain subtasks require manual work. First, the units of measurement and the coordinate system used by the indoor positioning system, the low-level controllers, and the Koord application program has to be matched. Secondly, the semantics of what it means for a robot to reach a point in space has to be decided and checked with implementations of low-level controllers and sensors. Our implementation of the quadrotor's low-level controller stipulates that a robot reaches a location (task) provided it enters a $r = 30$cm ball around the location and the indoor positioning system observes and transmits that fact. This value of the parameter r has to be decided by the developer.

V. Conclusion

In this paper we presented a programming package designed to simplify the development of distributed robotic applications. The software system interfaces with existing tools commonly used by roboticists providing easy integration on popular platforms by almost any user. In summary, our study suggests that Koord, Koord compiler, and Launcher can indeed enable users to develop and run distributed robotics applications in a hardware independent fashion; Koord programs can be ported across platforms automatically by linking with the right platform dependent libraries; The Simulator and KoordBMC tools can be used to simulate and verify the same application code. In the future, we plan to perform a user study to gain better understanding of the robot app developer's experience and efficiency with CyPhyHouse.

References

[1] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: an open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.

[2] Koord Language Development Team, "A design and analysis framework for distributed cps." [Online]. Available: https://cyphyhouse.github.io/papers/koord.pdf

[3] A. Nordmann, N. Hochgeschwender, and S. Wrede, *A Survey on Domain-Specific Languages in Robotics*. Cham: Springer International Publishing, 2014, pp. 195–206. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-11900-7_17

[4] C. Pinciroli, A. Lee-Brown, and G. Beltrame, "Buzz: An extensible programming language for self-organizing heterogeneous robot swarms," *CoRR*, vol. abs/1507.05946, 2015. [Online]. Available: http://arxiv.org/abs/1507.05946

[5] D. Zufferey, "The REACT language for robotics," 2017.

[6] M. Campusano and J. Fabry, "Live robot programming: The language, its implementation, and robot API independence," *Science of Computer Programming*, vol. 133, pp. 1 – 19, 2017.

[7] S. Gao, S. Kong, and E. M. Clarke, *dReal: An SMT Solver for Nonlinear Theories over the Reals*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 208–214. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-38574-2_14

[8] C. Fan, B. Qi, S. Mitra, and M. Viswanathan, "DryVR: Data-driven verification and compositional reasoning for automotive systems," in *Computer Aided Verification (CAV)*, July 2017.

[9] B. C. Williams, M. D. Ingham, S. H. Chung, and P. H. Elliott, "Model-based programming of intelligent embedded systems and robotic space explorers," *Proceedings of the IEEE*, vol. 91, no. 1, pp. 212–237, 2003.

[10] D. Pickem, P. Glotfelter, L. Wang, M. Mote, A. Ames, E. Feron, and M. Egerstedt, "The robotarium: A remotely accessible swarm robotics research testbed," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 1699–1706.

[11] B. Bohrer, Y. K. Tan, S. Mitsch, M. O. Myreen, and A. Platzer, "Veriphy: Verified controller executables from verified cyber-physical system models," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018. New York, NY, USA: ACM, 2018, pp. 617–630. [Online]. Available: http://doi.acm.org/10.1145/3192366.3192406

[12] B. Bohrer, V. Rahli, I. Vukotic, M. Völp, and A. Platzer, "Formally verified differential dynamic logic," in *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, ser. CPP 2017. New York, NY, USA: ACM, 2017, pp. 208–221. [Online]. Available: http://doi.acm.org/10.1145/3018610.3018616

[13] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, "Temporal-logic-based reactive mission and motion planning," *IEEE transactions on robotics*, vol. 25, no. 6, pp. 1370–1381, 2009.

[14] M. Kloetzer and C. Belta, "A fully automated framework for control of linear systems from temporal logic specifications," *IEEE Transactions on Automatic Control*, vol. 53, no. 1, pp. 287–297, 2008.

[15] T. Wongpiromsarn, U. Topcu, and R. M. Murray, "Receding horizon control for temporal logic specifications," in *Proceedings of the 13th ACM international conference on Hybrid systems: computation and control*. ACM, 2010, pp. 101–110.

[16] T. Wongpiromsarn, U. Topcu, N. Ozay, H. Xu, and R. M. Murray, "Tulip: a software toolbox for receding horizon temporal logic planning," in *Proceedings of the 14th international conference on Hybrid systems: computation and control*. ACM, 2011, pp. 313–314.

[17] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.

[18] B. Nitzberg and V. Lo, "Distributed shared memory: a survey of issues and algorithms," *Computer*, vol. 24, no. 8, pp. 52 –60, aug. 1991.

[19] S. V. Adve and K. Gharachorloo, "Shared memory consistency models: A tutorial," *IEEE Computer*, vol. 29, pp. 66–76, 1996.

[20] B. Calder, J. Wang, A. Ogus, N. Nilakantan, A. Skjolsvold, S. McKelvie, Y. Xu, S. Srivastav, J. Wu, H. Simitci, J. Haridas, C. Uddaraju, H. Khatri, A. Edwards, V. Bedekar, S. Mainali, R. Abbasi, A. Agarwal, M. F. u. Haq, M. I. u. Haq, D. Bhardwaj, S. Dayanand, A. Adusumilli, M. McNett, S. Sankaran, K. Manivannan, and L. Rigas, "Windows azure storage: A highly available cloud storage service with strong consistency," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, ser. SOSP '11. New York, NY, USA: ACM, 2011, pp. 143–157. [Online]. Available: http://doi.acm.org/10.1145/2043556.2043571

[21] "The Apache Cassandra project." http://cassandra.apache.org/, 2017.

[22] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles*, ser. SOSP '07. New York, NY, USA: ACM, 2007, pp. 205–220. [Online]. Available: http://doi.acm.org/10.1145/1294261.1294281

[23] A. Desai, V. Gupta, E. Jackson, S. Qadeer, S. Rajamani, and D. Zufferey, "P: Safe asynchronous event-driven programming," *SIGPLAN Not.*, vol. 48, no. 6, pp. 321–332, Jun. 2013. [Online]. Available: http://doi.acm.org/10.1145/2499370.2462184

[24] C. Drăgoi, T. A. Henzinger, and D. Zufferey, "Psync: A partially synchronous language for fault-tolerant distributed algorithms," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '16. New York, NY, USA: ACM, 2016, pp. 400–415. [Online]. Available: http://doi.acm.org/10.1145/2837614.2837650

[25] C. Muñoz, V. Carreño, and G. Dowek, *Formal Analysis of the Operational Concept for the Small Aircraft Transportation System*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 306–325.

[26] G. Rosu and T. F. Serbanuta, "K overview and simple case study," in *Proceedings of International K Workshop (K'11)*, ser. ENTCS, vol. 304. Elsevier, June 2014, pp. 3–56.

[27] B.-S. Chen and R. T. Yeh, "Formal specification and verification of distributed system." MARYLAND UNIV COLLEGE PARK DEPT OF COMPUTER SCIENCE, Tech. Rep., 1983.

[28] K. P. Birman, "The process group approach to reliable distributed computing," *Communications of the ACM*, vol. 36, no. 12, pp. 37–53, 1993.

[29] J. N. Tsitsiklis, "On the stability of asynchronous iterative processes," *Theory of Computing Systems*, vol. 20, no. 1, pp. 137–153, December 1987.

[30] V. Blondel, J. Hendrickx, A. Olshevsky, and J. Tsitsiklis, "Convergence in multiagent coordination consensus and flocking," in *Proceedings of the Joint forty-fourth IEEE Conference on Decision and Control and European Control Conference*, 2005, pp. 2996–3000.

[31] M. Mesbahi and M. Egerstedt, *Graph-theoretic Methods in Multiagent Networks*. Princeton University Press.

[32] R. G. Otap, "Development of a robotic testbed infrastructure with dynamic service discovery," Master's thesis, University of Illinois at Urbana-Champaign, 2012.

[33] MacGlashan, James, "Java ROSbridge."

[34] Mace, Jonathan and Lee, Jihoon and Toris, Russell and Alexander, Brandon and Bertam, David and Gruhler, Matthias, "Rosbridge suite."

[35] ArduPilot Development Team, "Ardupilot."

[36] Vladimir Ermakov, "mavros."